

Slidev Tutorial

From your first slide to advanced mastery — a hands-on guide, built with Slidev itself

Press Space to begin · o for the slide overview

What you'll learn

This tutorial is one continuous path, split into four tracks. Every feature is demonstrated live on the slide that explains it.



1 · Beginner

Install Slidev, write Markdown, master the built-in layouts.



2 · Intermediate

Click animations, live code, diagrams, math, icons, and media.



3 · Advanced

Custom layouts and components, theming, exporting, cheat sheet.



4 · Toolkit

The deck's own component library — and how to grow your own.



TIP

The whole tutorial is a Slidev deck. Press `o` at any time to see every slide at once and jump around — handy as a reference later.

■ PART 1

Beginner

Slidev fundamentals — from an empty folder to your first polished, running deck.

What is Slidev?

Slidev (slide + dev) is a presentation tool made for developers. You write your whole deck in a Markdown file, and Slidev renders it as a real web page powered by Vue and Vite.



Markdown-first

Your deck is plain text — fast to write, easy to diff, and friendly to version control.



Web-powered

Slides are HTML. Anything the web can do, your slide can do — including this tutorial.



TIP

You are reading a Slidev deck right now. Every feature in this tutorial runs live on the slide that explains it.

Installing pnpm

Slidev's docs use **pnpm** — a fast, disk-efficient package manager. Pick the line that matches your OS:

```
1  # macOS (Homebrew)
2  brew install pnpm
3
4  # macOS / Linux (install script)
5  curl -fsSL https://get.pnpm.io/install.sh | sh -
6
7  # Windows (PowerShell)
8  iwr https://get.pnpm.io/install.ps1 -useb | iex
9
10 # Already have Node 18+? Enable the bundled version
11 corepack enable
```

Restart your terminal, then check it works with `pnpm --version`.



NOTE

Prefer `npm` or `yarn`? They work too — swap `pnpm` for `npm` or `yarn` in any command in this tutorial.

Install and run

Scaffold a new project, then start the dev server:

```
1  # create a new deck – you'll be asked for a folder name
2  pnpm create slidev
3
4  # move into the project, then start it
5  pnpm dev
```

Slidev opens at **localhost:3030** with instant hot-reload: edit the Markdown, and the slide updates as you type.



TIP

If `pnpm` isn't found, see the previous slide to install it.

One file is your whole deck

A Slidev deck lives in a single `slides.md`. Slides are separated by `---` on its own line:

```
1  # First slide
2
3  Hello there!
4
5  ---
6
7  # Second slide
8
9  Welcome 🙌
```

Three dashes, and you're on the next slide. That simple rule is the entire structure of a deck.

Headmatter vs. frontmatter

Both use `---`, but they do different jobs. The **first** block configures the whole deck. **Every later** block configures only its own slide.

```
1  ---
2  theme: default      # headmatter – applies to the whole deck
3  title: My Talk
4  ---
5
6  # Opening slide
7
8  ---
9  layout: center      # frontmatter – applies to THIS slide only
10 ---
11
12 # A centered slide
```



HEADMATTER

The very first `---` block. Theme, title, fonts, transitions.



FRONTMATTER

Any later `---` block. Layout, classes, per-slide options.

Plain Markdown

Headings, **bold**, *italic*, lists, `inline code`, [links](#), blockquotes, and tables — all the Markdown you already know works on a slide.

The source on the left renders as the slide on the right.

```
1  ## A heading
2
3  - A bullet point
4  - With bold and `code`
5
6  > A short blockquote
```

A heading

- A bullet point
- With **bold** and `code`

A short blockquote

Layouts: a frame for each slide

A **layout** decides how a slide is arranged. Set it with the `layout:` frontmatter key — otherwise you get the `default` layout.

default

cover

center

two-cols

image-right

image-left

fact

quote

section

The next few slides each *use* one of these layouts, so you see exactly what it does.

This slide uses `layout: center`

Content sits in the middle of the screen — perfect for a single statement or a section break.

layout: two-cols

The slide body fills the **left** column.

Add a `::right::` marker, and everything after it flows into the **right** column.

```
1 ---
2 layout: two-cols
3 ---
4
5 # Left side
6
7 content here
8
9 ::right::
10
11 # Right side
12
13 content here
```

Picture beside content

The `image-right` layout puts an image on one side and your words on the other. Choose the picture with the `image:` key:

```
1 ---
2 layout: image-right
3 image: /slidev-logo.png
4 ---
```

Use `layout: image-left` to flip the image to the other side.



Slidev

1

The number of files you need to start a Slidev deck — your `slides.md`.

"The best deck is the one you can edit like code."

The Slidev philosophy

Speaker notes

End any slide with an HTML comment. It never appears on the slide itself — only in **presenter mode**, where you can read it while you talk.

```
1  # My slide
2
3  The content the audience sees
4
5  <!-- A private note, just for you, the speaker. -->
```



TIP

Open presenter mode at `localhost:3030/presenter`, or click the presenter icon in the navigation bar.

Getting around your deck

Space



Next animation or slide



Go back

o

Slide overview — see them all

f

Toggle fullscreen

d

Toggle dark mode

g

Jump to a slide by number or title



TIP

Hover the bottom-left corner of any slide to reveal a navigation bar with the same controls.



TIP

Press **g** then **Space** to list every slide by its title — or type any word from a title to filter, then **Enter** or click to jump. Handy on a long deck, where the **o** overview grid gets cramped.

The navigation bar

Hover the bottom-left corner of any slide. A toolbar appears, grouped by purpose — most buttons also have a keyboard shortcut.

Navigate

-  Fullscreen f
-  Previous / next slide
-  Slide overview o
-  Toggle light / dark d

Capture & annotate

-  Camera overlay in a corner
-  Screen recording
-  Drawing tools — annotate live

Modes & export

-  Enter presenter mode
-  Toggle editor · dev only
-  Browser exporter / PDF
-  Info dialog

Connect & settings

-  Presenter sync status
-  More options menu
- 18/70 Click to jump to a slide



TIP

The bar adapts to context: recording controls disappear in presenter mode, and the green sync arrows light up only when a presenter session is connected.

You can build a deck now

Install Slidev, write Markdown, pick a layout, and present. That's a complete workflow.

Next: make those slides **interactive**.

■ PART 2

Intermediate

Bring slides to life — click animations, live code, diagrams, math, and media.

Code blocks

Fence your code with a language, and Slidev highlights it with **Shiki** — the same engine that powers VS Code.

```
1  interface Slide {  
2    title: string  
3    layout?: string  
4  }  
5  
6  const intro: Slide = { title: 'Hello', layout: 'cover' }
```

You write it exactly like a fenced block anywhere in Markdown:

```
1  ```ts  
2  const x = 1  
3  ```
```

Highlight what matters

Turn on line numbers with `{lines:true}`, and spotlight specific lines by number:

```
1  function build() {  
2    const deck = loadSlides()  
3    render(deck)  
4    return deck  
5  }
```

`{2,4}` highlights lines 2 and 4. Ranges such as `{2-5}` work too.

Highlight on click

The very same block can walk through itself. Press Space to step the highlight:

```
1  const deck = loadSlides()  
2  render(deck)  
3  present(deck)
```

Separate line groups with | — each group is revealed by one click.

Editable code

Add `{monaco}` after the language and the block becomes a real editor. Type into it live during your talk:

```
const greet = (name: string): string => `Hello, ${name}!`  
  
greet('Slidev')
```



TIP

Great for "what if we change this?" moments — no switching to an editor.

Run code on the slide

`{monaco-run}` adds a **run** button — execute the code and show its output, right here:

```
function fib(n) {  
  return n <= 1 ? n : fib(n - 1) + fib(n - 2)  
}  
  
console.log([0, 1, 2, 3, 4, 5, 6, 7, 8, 9].map(fib))
```



```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Reveal content on click

Wrap any list in `<v-clicks>` and each item appears on its own click:

- First, frame the problem
- Then, show the approach
- Finally, reveal the result



TIP

For a single element, add the `v-click` directive directly to it.

Control the clicks

`v-click` — appear on the next click

`v-after` — appear together with the previous `v-click`

`v-click.hide` — start visible, then disappear on click

`v-click="3"` — appear at an exact click number

This... ..and this arrive together.

Move things with motion

The `v-motion` directive animates position, scale, and opacity as a slide opens:

→ I slide in from the left

```
1 <div v-motion :initial="{ x: -130 }" :enter="{ x: 0 }">
2   Hello
3 </div>
```

Slide transitions

This slide **fades** in — set per slide with the `transition:` key:

```
1  ---
2  transition: fade
3  ---
```

Built-in options include `fade`, `slide-left`, `slide-up`, and `view-transition`. Put one in the **headmatter** to make it the deck-wide default.

Math with LaTeX

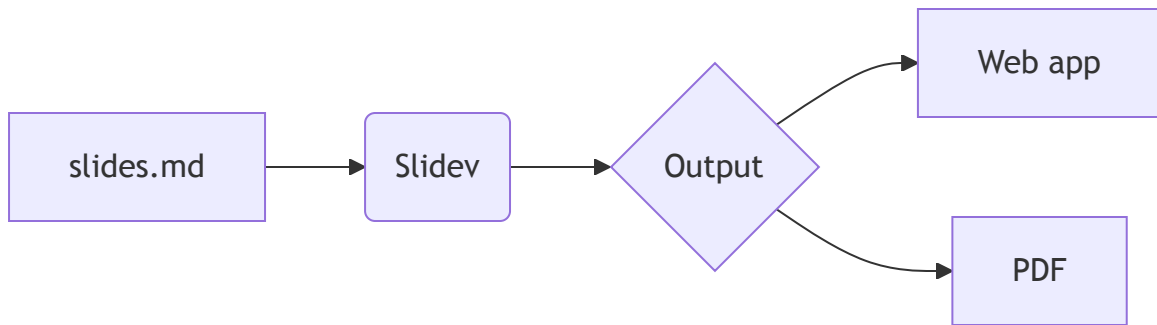
Inline math like $E = mc^2$ flows inside a sentence. Block math stands on its own:

$$\frac{\partial f}{\partial x} = 2x \qquad \int_0^1 x^2 dx = \frac{1}{3}$$

Wrap inline math in single `$`, and block math in double `$$`. Slidev renders it with KaTeX.

Diagrams from text

A fenced `mermaid` block becomes a diagram — no drawing tools required:



NOTE

``plantuml`` blocks work the same way, rendered through a PlantUML server.

Icons

Thousands of icons are available as components. Install an icon set, then drop them in inline:



```
1 <carbon-rocket /> &nbsp; <mdi-heart /> &nbsp; <logos-vue />
```

The component name is the icon set plus the icon, like `carbon-rocket`.

Images and media

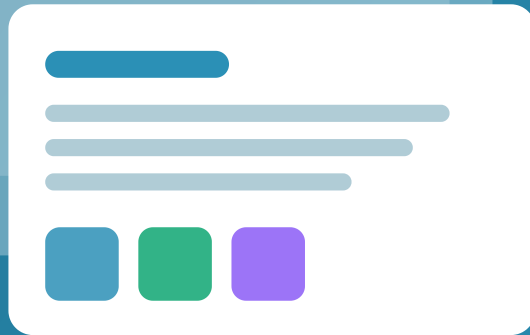
Keep image files in the `public/` folder and reference them from `/`.

Do not include `"/public"` in the reference although `public` is its parent folder.

```
1 ![A diagram](/slidev-logo.png)
```

For a full-slide backdrop, set the `background:` key in a slide's frontmatter.

The picture on the right is this very slide using `image-right` — the image *is* the demo.



Reuse slides with `src:`

Split a large deck across several files, then import each one:

```
1 ---
2 src: ./pages/01-beginner.md
3 ---
```



TIP

This tutorial does exactly that — three files (beginner, intermediate, advanced) stitched together into one `slides.md`.

Your slides are interactive now

Animations, live code, diagrams, math, and icons.

Next: make Slidev truly **your own**.

■ PART 3

Advanced

Make Slidev your own — custom layouts and components, theming, and an efficiency workflow.

Components in slides

Because slides are Vue, you can drop **components** straight into Markdown. Slidev ships many built-in ones:

`<Toc />`

`<Link />`

`<Tweet />`

`<Youtube />`

`<SlideCurrentNo />`

`<Link>` — in-deck navigation

```
1 <Link to="2">jump to overview</Link>
```

[jump to overview](#)

`<Youtube>` — embed a video by ID

```
1 <Youtube id="xuCn8ux2gbs"  
2   width="280" height="158" />
```



Write your own component

Drop a `.vue` file into `components/` and Slidev auto-imports it — use it by filename, no `import` needed.

```
1 <!-- components/Badge.vue -->
2 <template>
3   <span class="badge"><slot /></span>
4 </template>
```



TIP

Every `Callout`, `FeatureCard`, and `KeyCap` you've seen in this tutorial is a custom component living in `components/`.

Custom layouts

A layout is just a Vue component with a `<slot />` for the slide body. Put it in `layouts/`, then select it with `layout:`.

```
1 <!-- layouts/section.vue -->
2 <template>
3   <div class="section">
4     <slot />
5   </div>
6 </template>
```



TIP

The teal divider slides between each part of this tutorial use the deck's own `section` layout.

The setup/ directory

Files in `setup/` configure Slidev with code — no need to touch its internals:

`setup/main.ts` — extend the Vue app with plugins or components

`setup/shiki.ts` — customise the code highlighter

`setup/katex.ts` — add your own LaTeX macros

Each file exports a `define*` helper that Slidev calls at the right moment.

Styling with UnoCSS

Slidev bundles **UnoCSS** — atomic utility classes you apply right inside your Markdown. One class name = one CSS rule, so the styling lives on the element instead of in a separate file.

| The old way — write CSS

```
1 .title {  
2   padding-top: 1rem;  
3   font-size: 1.25rem;  
4   font-weight: 700;  
5 }
```

```
1 <div class="title">Hello</div>
```

| With UnoCSS — class names only

```
1 <div class="pt-4 text-xl font-bold">  
2   Hello  
3 </div>
```

You've seen these all tutorial long — `grid`, `flex`, `pt-4`, `text-lg`, `gap-3`. The next two slides are your reference for the ones you'll reach for most.

UnoCSS: spacing and layout

Spacing

- `p-4` — padding on all sides
- `pt-2 pb-4 pl-3 pr-3` — one side each
- `m-4` — margin (same idea)
- `gap-3` — space between grid/flex children

```
1 <div class="p-3 bg-slate-100">A padded box</div>
```

A padded box

Layout

- `flex` — make children sit in a row
- `grid grid-cols-2` — two-column grid
- `items-center` — vertical centering
- `justify-center` — horizontal centering

```
1 <div class="grid grid-cols-2 gap-4">  
2   <div>Left</div>  
3   <div>Right</div>  
4 </div>
```

Left

Right



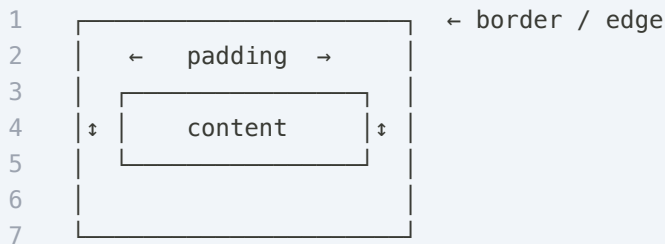
NOTE

Numbers are 4-pixel steps: `p-1` = 4px, `p-4` = 16px, `p-8` = 32px.

Padding vs margin — the box model

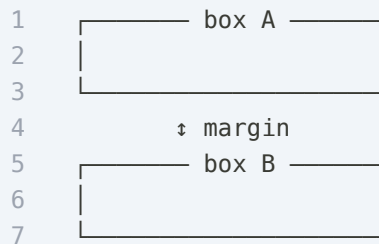
Padding is space *inside* a box, around its content. Margin is space *outside*, pushing neighbours away.

padding — inner space



- Pushes content **inward** from the edge.
- Background colour **fills** the padding.
- UnoCSS: `p-3`, `pt-3`, `px-4` ...

margin — outer space



- Pushes **other elements** away.
- Always **transparent** — no background.
- UnoCSS: `m-3`, `mt-3`, `-mt-12` (negative *pulls in*).



TIP

Mental model: **padding** = breathing room *inside*; **margin** = gap to neighbours *outside*; **border** = the line between them.

Flex, grid, and centring

Three primitives that lay out almost every slide.

flex

Children sit side-by-side in a **row**.

```
1 <div class="flex gap-2">
2   <div>A</div><div>B</div><div>C</div>
3 </div>
```

A B C

grid

Children fill **equal cells** in fixed columns.

```
1 <div class="grid grid-cols-3 gap-2">
2   <div>A</div><div>B</div><div>C</div>
3 </div>
```

A B C

items-center

Vertically aligns children of differing height.

```
1 <div class="flex items-center gap-2">
2   <div>tall<br>tall</div>
3   <div>short</div>
4 </div>
```

tall
tall short



TIP

Pair `items-center` with `justify-center` on a `flex` container — that's *perfect* centring, vertical and horizontal.

UnoCSS: typography and colour

| Typography

- `text-sm` / `text-base` / `text-lg` / `text-xl` / `text-2xl`
- `font-bold` — heavier weight
- `text-center` — centred text
- `leading-tight` — tighter line height

```
1 <div class="text-xl font-bold text-center">
2   Big, bold, centred
3 </div>
```

Big, bold, centred

| Colour

- `text-teal-500` — text colour
- `bg-slate-100` — background colour
- `opacity-60` — fade an element

Colours palette: a name + a shade from `50` to `900`.

```
1 <div class="text-teal-500 bg-slate-100 opacity-80">
2   Teal on slate, slightly faded
3 </div>
```

Teal on slate, slightly faded



TIP

Hover any class in the live playground at unocss.dev/interactive to see the exact CSS it emits.

UnoCSS: the naming pattern

Every spacing class is built from the same three pieces — read once, then write the rest from memory.

Anatomy of a class

— **m** **t** — **12**

- - optional — makes the value negative
- **m** p padding · **m** margin
- **t** side: **t** b l r, **x** (l+r), **y** (t+b), or none
- **12** step on the 0.25rem scale

The step scale

Each step adds **0.25rem** ($\approx 4\text{px}$), so the number stays small even at large sizes.

step	rem	px
1	0.25rem	4px
3	0.75rem	12px
4	1rem	16px



TIP

A negative margin like `-mt-16` pulls the next element upward — the trick for closing a gap left by a scaled-down card above.

When a slide overflows

Slidev slides are **fixed-size canvases** (1280×720), not scrollable pages. When content runs past the edge, you have two clean options.

1 · Shrink the whole slide

Add `zoom` to the slide's frontmatter:

```
1 ---
2 zoom: 0.85
3 ---
```

`0.7` – `0.95` is the sweet spot when content is just slightly too tall. Preferable for a talk — the audience never sees a scrollbar.

Need to shrink a *single* element instead of the slide? Reach for `transform scale-80 origin-top` — same idea as `zoom`, but localised.

2 · Scroll inside the slide

Wrap the overflowing region:

```
1 <div class="h-full overflow-y-auto">
2   ...long content...
3 </div>
```

Or scope scroll to one block with a fixed cap:

```
1 <div class="max-h-80 overflow-y-auto">...</div>
```



NOTE

For a defence or conference deck, reach for `zoom:` first. A scrollbar mid-slide tends to surprise the

Scoped slide styles

Need real CSS? Add a `<style>` block — it is scoped to **this slide only**:

This box is styled by a `<style>` block living on this very slide.

```
1 <style>
2 .demo-box { background: teal; color: white; }
3 </style>
```


The global context

Every slide can read Slidev's reactive state inside `{{ }}` expressions:

You are on slide **49** of **70**.

`$page` — the current slide number

`$clicks` — clicks made on this slide

`$nav` — navigate, count slides, jump around

`$frontmatter` — this slide's frontmatter

MDC: Markdown Components

With `mdc: true` in the headmatter, Markdown gets more expressive.

Add attributes inline — this word is **accented** using MDC syntax:

```
1 This word is [accented]{.text-teal-500 .font-bold} using MDC syntax.
```

Components also gain a block form, opened and closed with `::name::` fences — shown on the next two slides.

MDC in practice

Mix and match these in any Markdown slide:

Coloured span

```
1 [teal]{.text-teal-500}
```

→ teal

Link with classes

```
1 [sli.dev](https://sli.dev){.font-bold}
```

→ sli.dev

Heading with classes

```
1 ## Section {.text-teal-500}
```

Section

Click-revealed list

```
1 ::v-clicks
2 - First click reveals me
3 - Second click reveals me
4 ::
```

→ items appear one per click

Block components: Callout & FeatureCard

`::Component:: ... ::` blocks parse their body as Markdown — lists, links, and code all work inside:

Source:

```
1  ::Callout{type="tip"}
2  You can write a Callout in pure
3  Markdown — no Vue tags needed.
4
5  - bullets work
6  - inline `code` works
7  - so do [links](https://sli.dev) and bold
8  ::
```

```
1  ::FeatureCard{title="Fast" icon="i-carbon-rocket"
2    row}
3  Boots in under a second.
4  ::
```

Renders as:



TIP

You can write a **Callout** in pure Markdown — no Vue tags needed.

- bullets work
- inline `code` works
- so do [links](https://sli.dev) and **bold**



Fast

Boots in under a second.



NOTE

Reach for MDC when the content inside a component is still Markdown. Reach for plain HTML/JSEX when it isn't.

Themes

A **theme** restyles your whole deck. Switch with a single line of headmatter:

```
1  ---
2  theme: seriph
3  ---
```

Slidev installs the theme for you on the next run. Browse the gallery at sli.dev/resources/theme-gallery, or build your own from a custom layout set.

Addons

Addons bundle features — components, layouts, behaviours — to share across decks:

```
1  ---
2  addons:
3    - slidedev-addon-tldraw
4  ---
```

Where a *theme* changes how the deck **looks**, an *addon* changes what it can **do**.

Presenter mode

A dedicated view for speaking: the current slide, a preview of the next one, your notes, and a timer — all at once.



TRY IT YOURSELF

Run the dev server and open `localhost:3030/presenter`, or click the presenter icon in the navigation bar.

Write notes as an HTML comment at the end of a slide, and they appear here beside it.

Draw, annotate, and appear

Slidev has a built-in **drawing toolbar** — sketch on your slides while presenting, and the annotations are saved with the deck.



TRY IT YOURSELF

Hover the navigation bar, click the pen icon to open the drawing tools, then sketch right on this slide.

You can also show your **camera** in a corner and **record** the talk — both from that same navigation bar.

Exporting your deck

Turn the deck into a shareable file:

`slidev export` — a PDF

`slidev export --format pptx` — a PowerPoint file

`slidev export --format png` — one image per slide



NOTE

Exporting needs the `playwright-chromium` package — Slidev prompts you to install it the first time.

Deploying as a website

`slidev build` produces a static site in `dist/` that you can host anywhere:

```
1  slidev build
```

Drop the `dist/` folder onto Netlify, Vercel, or GitHub Pages. For a sub-path URL, set the base path:

```
1  slidev build --base /my-talk/
```

Efficiency cheat sheet

While presenting

`Space` next · `←` back

`o` overview · `g` go to slide

`f` fullscreen · `d` dark mode

Pen icon in the nav bar — draw

While building

Edit Markdown — hot reload is instant

`/presenter` — the speaker view

`components/` — your components

`layouts/` — your layouts

`slidev export` — PDF

`slidev build` — static site

Common Carbon icons

A pocket reference. Use any of these as a component (`<carbon-rocket />`) or a UnoCSS class (`i-carbon-rocket`). Browse the full set at icones.js.org.



rocket



code



tools



idea



edit



pen



search



settings



chart-line



dashboard



analytics



data-base



cloud



api



document



folder



email



chat



notification



user-avatar



group



calendar



time



light



checkmark



close



warning



information



favorite



star



download



share

■ PART 4

Toolkit

The deck's own component library — what it is, and how to grow your own.

Why a deck has a toolkit

When the same small pattern appears on several slides, give it a name. This deck does:



Components

Drop a `.vue` file in `components/` — Slidev auto-imports it. Use it by filename, anywhere in your Markdown.



Layouts

Drop a `.vue` file in `layouts/` — select it per slide with `layout:`.

`<Callout>` — coloured highlight box

Inline emphasis with a Carbon icon and a label. Four semantic types — plus `blank` for an aside that doesn't fit a category.



TIP

Keep slides to one idea each.



WATCH OUT

This step needs a dependency.



NOTE

Speaker view lives at `/presenter`.



TRY IT YOURSELF

Press `o` right now.

No icon, no label — just a neutral rail for a highlight that isn't tip, warning, note, or try.

```
1 <Callout type="tip">Keep slides to one idea.</Callout>
2 <Callout type="warning" title="Heads up">Optional custom title.</Callout>
3 <Callout type="blank">Neutral highlight, no label.</Callout>
```

`<Chip>` — a labelled pill

A monospace pill for naming layouts, features, keys — anything you want to call out without a full sentence.

default

cover

center

two-cols

image-right

fact

```
1 <div class="flex flex-wrap gap-2">
2   <Chip>default</Chip>
3   <Chip>cover</Chip>
4   <Chip>center</Chip>
5 </div>
```



NOTE

`<Chip>` only styles itself — wrapping a row of them in `flex flex-wrap gap-2` is the parent's job. Same idea as `<KeyCap>`. For a comma-separated row, reach for `<Chips>`.

`<Chips>` — pills from a comma-separated list

Put your labels in the slot, separated by commas. Chips renders one `<Chip>` per item — the plural of `<Chip>`, already laid out in a wrapping row:

default

cover

center

two-cols

image-right

fact

```
1 <Chips>default, cover, center</Chips>
```



TIP

`<Chip>` is the single pill; `<Chips>` is the comma-list shorthand. Same relationship as `<KeyCap>` and `<KeyCaps>`.

<KeyCaps> — keycaps from a comma-separated list

Put your keys in the slot, separated by commas. KeyCaps renders one keycap per key — the plural of <KeyCap>. Whatever label you want goes right after the tag:

o Slide overview

f Toggle fullscreen

Space → Next animation or slide

d Toggle dark mode

```
1 <KeyCaps>o</KeyCaps> Slide overview
2 <KeyCaps>Space,→</KeyCaps> Next animation or slide
```



TIP

For a single key, <KeyCaps>o</KeyCaps> and <KeyCap>o</KeyCap> are equivalent. KeyCaps earns its keep when the list has more than one.

layouts/multicolumns.vue — this slide is using it

A four-column reference layout. Define `#col1`–`#col4`; the layout counts the filled ones and picks the grid. Two columns? It draws two. Four? It draws four.

Define

```
1  ---
2  layout: multicolumns
3  ---
4
5  # Heading
6
7  <template #col1>...</
8  <template #col2>...</
```

Navigate

- o Overview
- f Fullscreen
- d Dark mode

Style

- `--deck-accent`
— accent colour
- `--deck-accent-deep` — gradient
deep
- `--deck-radius`
— corner radius
- Carbon icons via
`i-carbon-*`

Compose

Chip Chips
KeyCaps Callout
FeatureCard

Build your own

When a pattern shows up three times, name it. The recipe is short:



A component

Add a `.vue` file to `components/`. Use it by filename anywhere in your Markdown — Slidev auto-imports it.



A layout

Add a `.vue` file with a `<slot />` to `layouts/`. Select it on a slide with the `layout:` frontmatter key.



TRY IT YOURSELF

Open `components/Chip.vue` as a starting template — a header comment, a slot, and a handful of CSS rules. Copy it, rename it, and you have your own.

You have a toolkit now

Three new pieces, all extracted from patterns that were already in this deck.

Reach for them on the next slide you write — or grow your own.

You've mastered Slidev

From your first `---` separator to custom layouts, live code, and a deployed site.

[Documentation](#)

[Themes](#)

[Showcases](#)

Now go build something worth presenting.